PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

High Speed Link Lite

HBM Base Die Digital Link Slice — Spec · Architecture · RTL · Verification · PD Guide

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	권용현 · 010-6408-8646
저장소	`github.com/yhkwon-git/High_Speed_Link_Lite` · `commit 9d579bc` · `p.3–20 측정 4feb39f`
사용 도구	SystemVerilog · Verilator 5.052 · ModelSim 10.1d · Yosys 0.52 · OpenSTA 3.1 (Nangate45) · Python

※ p.3–20의 파형 · 커버리지 · 합성 수치는 commit 4feb39f를 수정 없이 재실행해 측정 (2026-10-30). p.21–22는 이후 반영분(리뷰 수정 d1f8e2b, DBI-AC e30ea96), p.23–28은 DFT / DFD 확장(f018966 → 14fec6e, 쪽마다 측정 commit), p.2의 회귀 · 코드 규모는 1361a29 기준.

HBM Base Die 디지털 링크 Slice 설계 개요

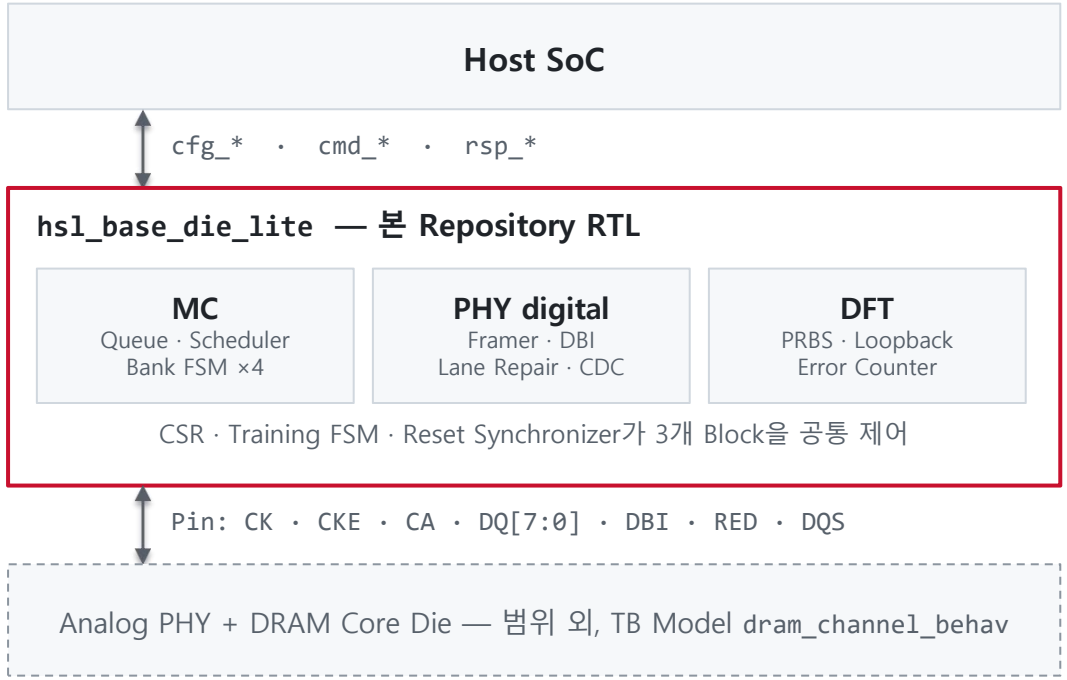
HBM Base Die 1개 Channel을 Study Scale로 축소 — Spec 정의부터 PD Guide까지 단독 수행 가능한 범위로 구성

배경 및 목표

- **Custom HBM 전제** (Memory Controller를 Base Die로 이동, 표준 HBM은 Host SoC 측) — Base Die가 Command Scheduling · Training · Data Encoding · Test를 디지털로 중재
- 고속 PHY는 Analog 비중이 높아, 디지털 담당 영역의 **경계** (Pin 논리값 · Handshake · CDC · Reset · DFT 관측점)을 문서와 RTL로 명확히 정의하는 것이 목표
- 실제 HBM Channel 폭 대신 DQ 8 · BL 4 · Bank 4로 축소, MC | PHY Digital | DFT 구조와 설계 규칙은 동일하게 유지

Out of Scope — docs/01_requirements.md

Multi-channel · 전력(mW) Sign-off · Analog Margin · JEDEC/DFI 호환 선언 — 근거 없는 수치는 기재하지 않음



MC · PHY · DFT

설계 범위 · 목표 3.2 Gb/s/DQ (Controller 800 MHz / PHY 1.6 GHz)

21 모듈 · 1,542줄

RTL 22파일 1,953줄 (주석·빈 줄 제외 1,542줄) · TB 1,681줄 (HEAD)

15 / 15 PASS

SMOKE · T1–T9 · E7 · E9 · E11 · unit 단일 Regression (Verilator 5.052 · ModelSim 10.1d, HEAD)

87.5% · 96.6%

Line · Branch Coverage (Toggle 56.6%, commit 4feb39f)

요구사항 정의 및 주요 Spec

Spec 변경 순서: docs/01_requirements.md → hsl_pkg · RTL에만 존재하는 수치는 사용하지 않음

기능 요구사항

1. Training 완료 전 Host Command 수신 차단
2. 순서: RESET → CKE → MRS → ZQ → WRLVL → RDLVL → DESKEW → WRTRAIN → MISSION
3. MISSION 이후 ACT · PRE · RD · WR · REF 수신 (REF는 Encoding만 정의, 미발행)
4. RD Response는 Burst 1개, 준비 전 `rsp_valid` = 0
5. Loopback: TX Burst를 RX로 반환
6. Lane Repair: DQ 1개를 Redundant Pin(RED)으로 대체
7. ID · Version · Training State · Error Count Read 지원

주요 Parameter (hsl_pkg)

항목	값
DQ / DBI / RED	8 / 1 / 1 → PIN_W 10
Burst	BL 4 Beat (Beat 0 = 첫 UI)
Bank · Row · Col	4 · 8 bit · 4 bit
Data 폭	32 bit = DQ_W × BL
Target Data Rate	3.2 Gb/s/DQ → raw 3.2 GB/s
Clock	ctrl 1.25 ns · PHY 0.625 ns

Power · Reliability 규칙

- CKE는 FSM이 RESET일 때만 0
- DBI: beat의 1 개수 > DQ_W/2 이면 반전
- Driver OE는 Burst 구간 외 0
- Error Counter 16 bit Saturation, Wrap-around 없음

Timing 기본값 (ctrl Clock 단위)

Parameter	Clock	담당 Block
`tRCD`	4	bank FSM
`tRP`	4	bank FSM
`tRAS`	10	bank FSM
`tRTP`	3	bank FSM
`tWR`	4	bank FSM
`tCCD`	2	scheduler
`tWTR`	4	scheduler
`tRFC`	20	(REF 미발행)

Acceptance Criteria

- `[SMOKE] PASS` 유지 (신규 Test 추가 후에도)
- `ready` = 0 구간 Command Drop 시 Fail
- Timeout은 Fail, X 값 Data Sampling 금지

Clock 구조 Trade-off 검토 및 Architecture 선정

후보 3안을 성능 · 전력 · 면적 · Risk 기준으로 비교 — UI Serialization(C)은 Analog PHY 영역으로 분리 (docs/02_architecture.md)

	A. Single Clock · Burst 폭	B. Controller 1 : PHY 2	C. RTL UI Serialization
성능	Bank Timing · Scheduling 효율	A + Gearbox Throughput	Waveform 상 Eye 관찰 용이
전력	DBI · CKE · OE	A와 동일, Clock 1개 추가	Pin 단위 Toggle로 DBI 효과 확인 용이
면적	32 bit Datapath	16 bit PHY Beat + 소형 FIFO	Shift Register는 작으나 검증 시간 증가
Risk	CDC 미적용	Pointer 동기화 Bug	Simulation 속도 저하, Analog 영역과 혼동 우려
결정	**v0.1 출발점** — Handshake · Bank Timing 우선 검증	**E7에서 전환** — `clk_phy` · Gearbox · Async FIFO	미채택 — Pin UI Serialization은 Analog PHY 영역

A → B 전환 시 Sim Clock · SDC · `hsl_pkg` 상수를 동일 수식으로 재정렬, 1건이라도 불일치 시 미완료로 보는 Cross-check Table(E12) 작성

Clock 수식 정의

CK = 3.2 Gb/s ÷ 2 = 1.6 GHz (625 ps), Controller = CK ÷ 2 = 800 MHz. beats_per_burst = BL ÷ DDR_BITS = 2 → Controller 1 Cycle당 32b Burst 1개

In-order Scheduler

Queue Head만 Issue — 타 Bank가 Open이어도 대기하므로 Bandwidth는 저하되나 검증 · 면적은 단순. Bank FSM 분리로 OoO 전환 시에도 Queue 구조 재사용

Beat-major Data 배치

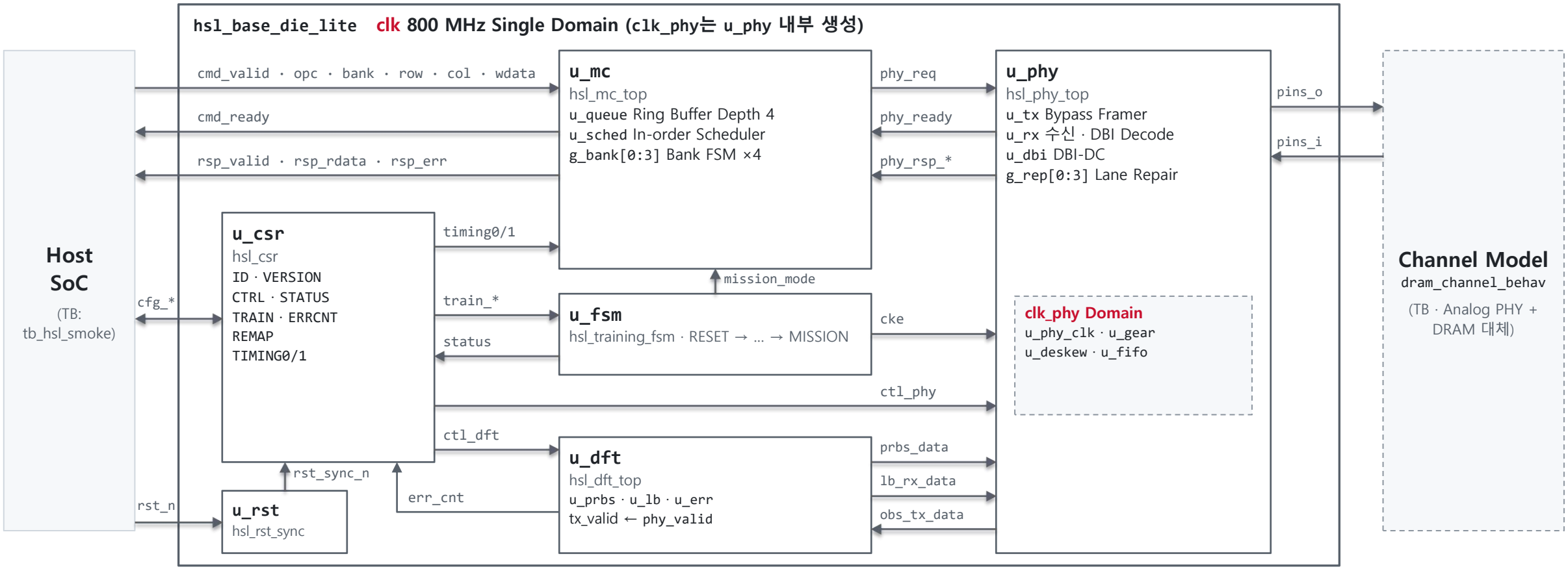
beat b = `data[b*DQ_W +: DQ_W]`. DBI · ECC는 1 UI의 Byte 단위로 처리 — Lane 단위 저장 시 Bit 분산 Access 필요

Reset Domain 분리

CSR은 `rst_sync_n`만, Datapath는 `rst_sync_n & ~soft_rst`. CSR까지 포함 시 soft_rst Bit가 자체 Clear되어 Reset 유지 불가

Top Level 구성 및 Reset · Clock 구조

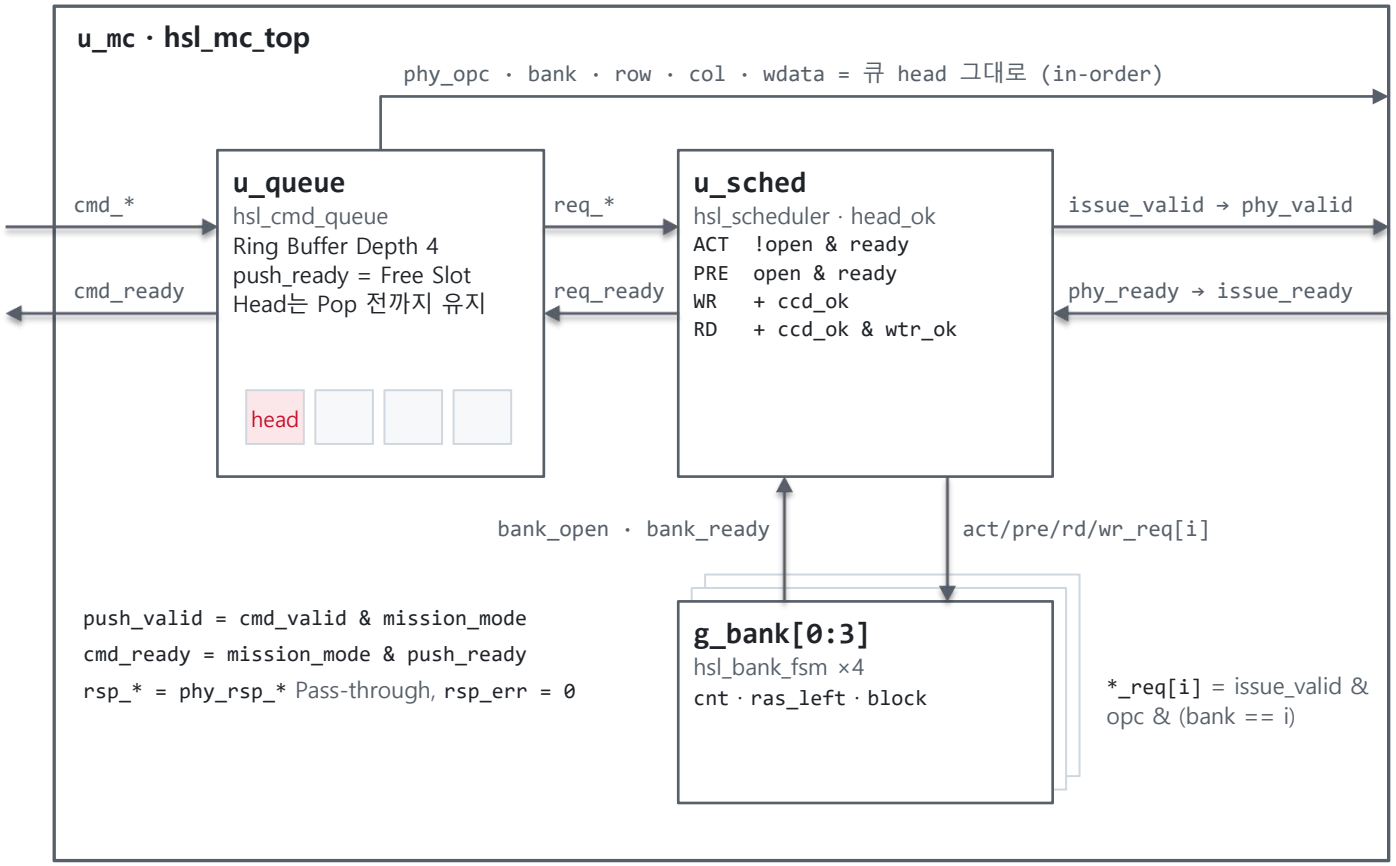
rtl/top/hsl_base_die_lite.sv 기준 6개 Instance · 단일 Reset Tree · Controller Clock 단일 Domain (명칭은 RTL과 동일)



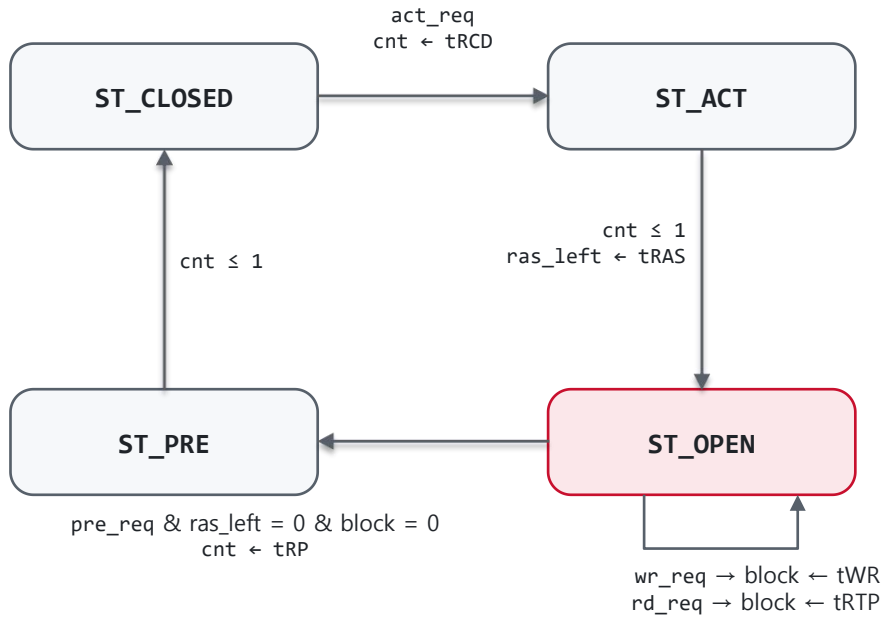
Legend `phy_req` = phy_valid, phy_opc/bank/row/col/wdata · `status` = train_state, train_done, mission_mode · `train_*` = train_bypass/start/wait, zq/wrlvl/rdlvl/deskew_done
`ctl_phy` = dbi_en, phy_bypass, repair_en/lane, loopback, prbs_en · `ctl_dft` = loopback, prbs_en, err_clear · `pins_o` = pin_ck/cke/ca/dq_o[9:0]/dq_oe/dqs_o · `pins_i` = pin_dq_i[9:0], pin_dqs_i
Reset: `u_csr`는 `rst_sync_n`, 그 외 4개 Block은 `rst_logic_n = rst_sync_n & ~soft_rst` (Top assign 1줄)

Memory Controller 구조 및 Timing 제어 분담

Queue → In-order Scheduler → Bank FSM ×4 · tRCD · tRP · tRAS · tWR · tRTP는 Bank FSM, tCCD · tWTR는 Scheduler 담당



hsl_bank_fsm State Diagram (Bank별 1개)

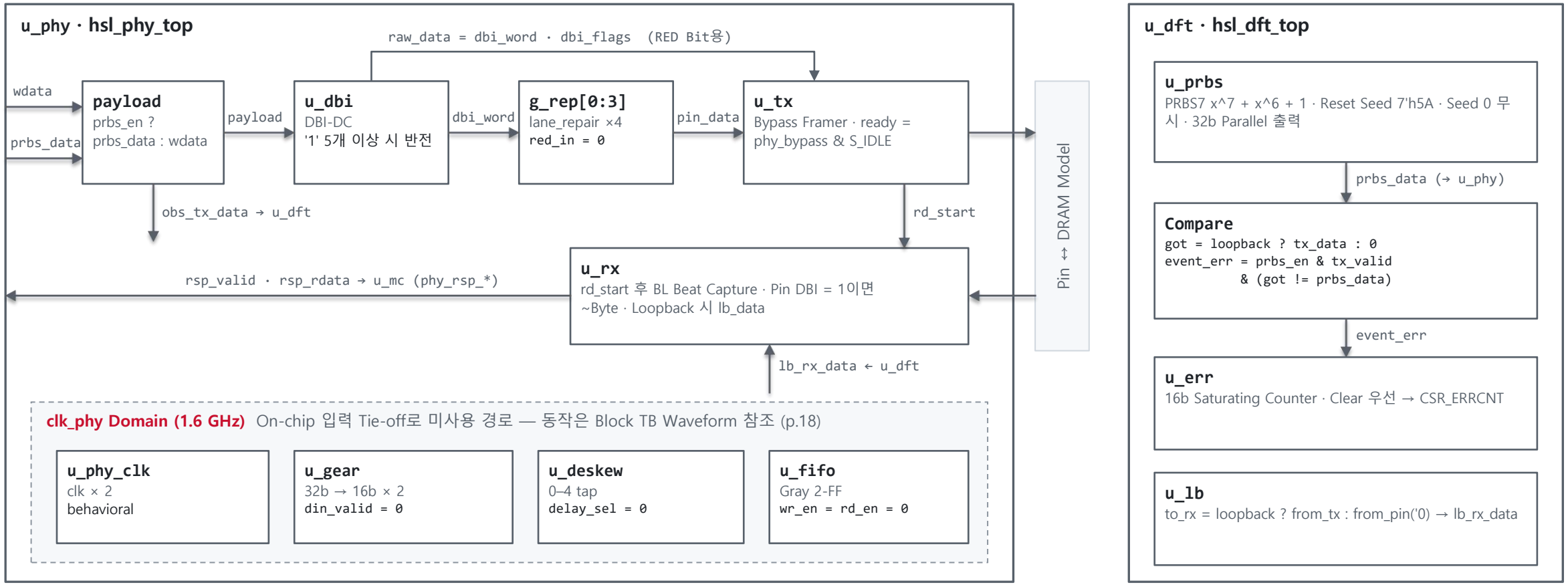



```
timing_ok = ST_CLOSED | (ST_OPEN & ras_left == 0 & block == 0)
is_open   = ST_OPEN
```

Note Column Command(RD/WR)도 `ras_left = 0`까지 대기 → T4 ACT → WR Issue 간격 15 Cycle. tRCD 이후 Column을 허용하는 JEDEC 대비 보수적인 Study 규칙

PHY Datapath 및 DFT 구조

Bypass Framer가 Pin 구동 · On-chip Gearbox · FIFO · Deskew는 입력 Tie-off 상태로 tb_hsl_blocks 독립 Instance에서 검증



`tx_valid = phy_valid` (MC → PHY Command Issue Cycle마다 Compare) · Pin Loopback은 Compare 대상 제외 (docs/06_dft_plan.md) · RX는 RED Pin 미사용으로 Lane Repair는 TX Pin Level까지만 검증

Reset Scheme 및 Training Sequence 구현

Async Assert · Sync Deassert (2-FF) · CSR은 rst_sync_n, 그 외 Block은 rst_logic_n · FSM은 done Pulse 수신 전 State 유지

rtl/top/hsl_rst_sync.sv L19–27

```
19  always_ff @(posedge clk or negedge rst_ni) begin
20    if (!rst_ni) begin
21      sync_r <= 2'b00;
22    end else begin
23      sync_r <= {sync_r[0], 1'b1};   ❶
24    end
25  end
26
27  assign rst_no = sync_r[1];
```

rtl/top/hsl_base_die_lite.sv L73, L75–77, L105–107

```
73  assign rst_logic_n = rst_sync_n & ~soft_rst;   ❷
    ----------------------------------------------
75  hsl_csr u_csr (
76    .clk          (clk),
77    .rst_n        (rst_sync_n),   ❸
    ----------------------------------------------
105 hsl_training_fsm u_fsm (
106   .clk          (clk),
107   .rst_n        (rst_logic_n),
```

rtl/ctrl/hsl_training_fsm.sv L38–40, L42–57

```
38  assign train_done   = (state == TR_MISSION);
39  assign mission_mode = train_done;
40  assign cke          = (state != TR_RESET);   ❹
    ----------------------------------------------
42  always_ff @(posedge clk or negedge rst_n) begin
43    if (!rst_n) begin
44      state <= TR_RESET;
45    end else if (train_bypass) begin   ❺
46      state <= TR_MISSION;
47    end else begin
48      case (state)
49        TR_RESET:   if (train_start) state <= TR_CKE;
50        TR_CKE:     state <= TR_MRS;
51        TR_MRS:     state <= TR_ZQCAL;
52        TR_ZQCAL:   if (zq_done) state <= TR_WRLVL;   ❻
53        TR_WRLVL:   if (wrlvl_done) state <= TR_RDLVL;
54        TR_RDLVL:   if (rdlvl_done) state <= TR_DESKEW;
55        TR_DESKEW:  if (deskew_done) state <= TR_WRTRAIN;
56        TR_WRTRAIN: state <= TR_MISSION;
57        TR_MISSION: state <= TR_MISSION;
```

❶ `sync_r <= {sync_r[0], 1'b1}` — Assert는 비동기, Deassert는 clk Rising 2회 후. Raw Pin은 타 Block에서 직접 사용하지 않음

❷ `rst_logic_n = rst_sync_n & ~soft_rst` — Soft Reset은 MC · FSM · PHY · DFT만 초기화

❸ CSR은 `rst_sync_n`만 사용 → CTRL.soft_rst Bit 자체 Clear 방지, Reset 상태 유지 가능

❹ `cke = (state != TR_RESET)` — RESET State에서만 CKE 0, Smoke Test에서 Pin으로 확인

❺ `train_bypass`가 case문보다 우선 → 1 Cycle 내 MISSION 진입. T4–T8은 이 경로로 진입

❻ ZQCAL · WRLVL · RDLVL · DESKEW는 done Pulse 수신 전까지 State 유지 (T1)

검증 항목 SMOKE — Reset 직후 RESET · CKE 0 · cmd_ready 0, Bypass 후 MISSION, soft_rst 중 RESET 후 복귀 (expect_eq 27곳) · T1 — done Pulse 대기 및 State 순서 · Waveform p.14 · 15

In-order Scheduler 및 Bank Timing 제어

req_ready는 head_ok와 PHY ready로만 생성해 Command Drop 방지 · Bank Timing은 timing_ok 1 bit로 집약

rtl/mc/hsl_scheduler.sv L35-38, L43-51, L56-57

```
35  wire ccd_ok = (ccd_cnt == 8'd0);
36  wire wtr_ok = (wtr_cnt == 8'd0);
37
38  wire act_ok = !bank_open[req_bank] && bank_ready[req_bank];  (1)
    ----------------------------------------------------------------
43    head_ok = 1'b0;
44    if (req_valid) begin
45      case (req_opc)
46        OPC_NOP: head_ok = 1'b1;
47        OPC_ACT: head_ok = act_ok;
48        OPC_PRE: head_ok = col_ok;
49        OPC_RD:  head_ok = col_ok && ccd_ok && wtr_ok;  (2)
50        OPC_WR:  head_ok = col_ok && ccd_ok;
51        default: head_ok = 1'b0;  (3)
    ----------------------------------------------------------------
56  assign req_ready   = head_ok && issue_ready;  (4)
57  assign issue_valid = req_valid && req_ready;
```

rtl/mc/hsl_bank_fsm.sv L48-50, L75-87

```
48  assign is_open   = (st == ST_OPEN);
49  assign timing_ok = (st == ST_CLOSED) ||
50                     ((st == ST_OPEN) && (ras_left == 8'd0) && (block == 8'd0));  (5)
    ----------------------------------------------------------------
75        ST_OPEN: begin
76          if (ras_left != 8'd0) begin  (6)
77            ras_left <= ras_left - 8'd1;
78          end else if (block != 8'd0) begin
79            block <= block - 8'd1;
80          end else if (pre_req) begin
81            st  <= ST_PRE;
82            cnt <= t_rp_u;
83          end else if (wr_req) begin  (7)
84            block <= t_wr;
85          end else if (rd_req) begin
86            block <= t_rtp;
87          end
```

1. act_ok · col_ok: Bank의 is_open · timing_ok만 참조 (Bank Timing은 Bank FSM 담당)
2. RD는 tCCD · tWTR, WR는 tCCD 추가 Check (Scheduler 담당 Counter)
3. REF · Reserved Opcode는 default → head_ok = 0. Head 도달 시 Queue Stall (p.19 D)
4. req_ready = head_ok && issue_ready — PHY Busy 시 Pop하지 않아 Command Drop 없음
5. timing_ok: CLOSED 또는 OPEN이면서 ras_left와 block이 모두 0
6. OPEN State에서 tRAS Countdown 우선 — Column Command도 tRAS 이후 (T4: ACT → WR 15 Cycle)
7. WR는 block ← tWR, RD는 block ← tRTP — 후속 PRE · Column Command Block

※ 4feb39f 코드 · Gray 크로싱은 d1f8e2b에서 수정 → p.22

Gray Pointer Async FIFO 및 Toggle Handshake Gearbox

Write clk_phy → Read clk · Pointer는 Gray Code 2-FF 동기화 · Gearbox는 Toggle 1쌍당 32b Word 1개 전달

rtl/phy/hsl_cdc_fifo.sv L40-42, L44-47, L65, L70-75

```
40  function automatic logic [AW:0] bin2gray(input logic [AW:0] b);
41    bin2gray = (b >> 1) ^ b;                                          ①
42  endfunction

44  assign wgray    = bin2gray(wbin);
45  assign rgray    = bin2gray(rbin);
46  assign wr_full  = (wgray == {~rptr_g_s2[AW], ~rptr_g_s2[AW-1], rptr_g_s2[AW-2:0]});  ②
47  assign rd_empty = (rgray == wptr_g_s2);

65  always_ff @(posedge clk_r or negedge rst_n) begin

70    end else begin
71      wptr_g_s1 <= wgray;                                             ③
72      wptr_g_s2 <= wptr_g_s1;
73      if (rd_en && !rd_empty) begin
74        rbin <= rbin + 1'b1;
75      end
```

① bin2gray: Pointer를 Gray Code로 전달해 1 bit씩만 변화 → 동기화 중 Mis-sampling 시에도 인접 값

② Full은 상위 2 bit 반전 비교, Empty는 동기화된 Write Pointer와 일치 시

③ Read Clock의 `wptr_g_s1 → s2` 2-FF. SDC는 False Path 대신 `set_max_delay 1.25`

rtl/phy/hsl_gearbox.sv L35, L46-48, L63-68, L74-78

```
35  assign din_ready = (req_tog == ack_s2);                             ④

46      if (din_valid && din_ready) begin
47        hold    <= din_ctrl;
48        req_tog <= ~req_tog;

63      req_s1 <= req_tog;
64      req_s2 <= req_s1;
65      if (!beat) begin
66        if (req_s2 != ack_tog) begin                                  ⑤
67          shift      <= hold;
68          dout_phy   <= hold[DQ_W*DDR_BITS-1:0];

74      end else begin
75        dout_phy   <= shift[DATA_W-1:DQ_W*DDR_BITS];
76        dout_valid <= 1'b1;
77        beat       <= 1'b0;
78        ack_tog    <= ~ack_tog;                                       ⑥
```

④ `din_ready = (req_tog == ack_s2)` — Toggle 1쌍당 Word 1개

⑤ PHY Clock에서 `req_s2 != ack_tog`이면 하위 16b부터 출력

⑥ 두 번째 Beat 후 `ack_tog` Toggle → ctrl측 2-FF 후 다음 Word

※ DC 규칙 · DBI-AC 추가 → p.21

DBI-DC · Lane Repair · Saturating Error Counter

Pin High 수를 줄이는 DBI · Bad Lane을 RED로 우회하는 Lane Repair · Wrap-around 없는 16b Error Counter

rtl/phy/hsl_dbi.sv L17–33

```
17  function automatic logic [3:0] pop8(input logic [7:0] b);
18    pop8 = {3'b0, b[0]} + b[1] + b[2] + b[3] + b[4] + b[5] + b[6] + b[7];  (1)
19  endfunction
20
21  integer bi;  (2)
22  always_comb begin
23    dout = din;
24    dbi  = '0;
25    if (en) begin
26      for (bi = 0; bi < BL; bi = bi + 1) begin
27        if (pop8(din[bi*DQ_W +: DQ_W]) > 4'd4) begin  (3)
28          dout[bi*DQ_W +: DQ_W] = ~din[bi*DQ_W +: DQ_W];
29          dbi[bi] = 1'b1;
30        end
31      end
32    end
33  end
```

(1) pop8: Beat당 '1' 개수 (4 bit 합)

(2) Module Scope `integer bi` — Verilator 통과, Yosys 0.52는 Latch로 판정 (p.20)

(3) '1'이 4개 초과 시 Beat 반전 + Flag → Pin High 수 ≤ DQ_W/2

rtl/phy/hsl_phy_tx.sv L63–68, L136–138

```
63  function automatic logic red_of(input logic [2:0] beat);
64    if (repair_en && (repair_lane < 4'd8)) begin
65      red_of = raw_r[beat*DQ_W + 32'(repair_lane)];  (4)
66    end else begin
67      red_of = 1'b0;
68    end
    ----------------------------------------------------------------
136       S_WD: begin
137         ca    <= {2'b00, bank_r, OPC_WR};
138         dq_o  <= {red_of(idx), dbi_r[idx[1:0]], data_r[idx*DQ_W +: DQ_W]};  (5)
```

(4) `red_of`: Repair Lane의 원래 Bit (DBI 적용 후 `raw_r`), Lane ≥ 8이면 0

(5) Data Beat Pin = {RED, DBI, DQ[7:0]}. DQ[lane]은 `g_rep`가 0으로 대체

rtl/dft/hsl_err_cnt.sv L22–25

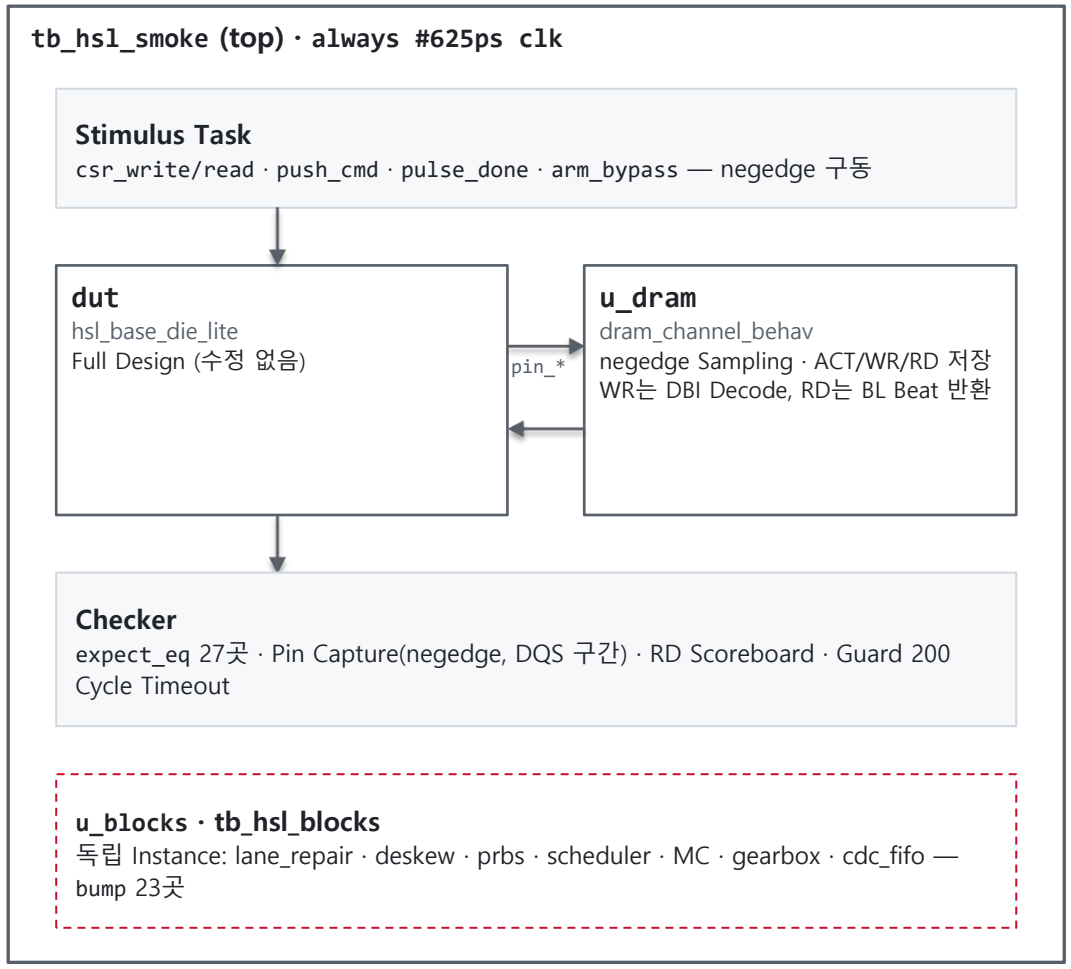
```
22    end else if (clear) begin  (6)
23      count <= '0;
24    end else if (event_valid && event_err && (count != 16'hFFFF)) begin  (7)
25      count <= count + 16'd1;
```

(6) `clear`가 동일 Cycle Error보다 우선

(7) `16'hFFFF`에서 Saturation — Fail 지속 시 Wrap-around로 정상처럼 보이는 현상 방지

검증 환경 구성 및 Test Plan

Smoke · Block · Channel 3계층 단일 Regression · tb_hsl_smoke가 tb_hsl_blocks 포함 · [ALL] PASS 외에는 Non-zero Exit Code



ID	대상	Check 항목	위치
SMOKE	기본 동작	ID · VERSION · TIMING Reset 값, 미정의 Address 0, Reset 직후 Pin · ready 0, Bypass → MISSION, Depth 4 Full → ready 0, soft_rst	smoke
T1	Training	순차 진행, done Pulse 전 State 유지	smoke
T2	Bank FSM	tRCD 전 RD 차단, tRP 전 ACT 차단	blocks
T3	Scheduler	tCCD 전 후속 Column Command 차단	blocks
T4	Channel	ACT → WR → RD, A5A5_5A5A 일치	smoke
T5	DBI	0x1F → Pin E0 + DBI 1, 0x0F → 유지	smoke
T6	CDC FIFO	2 Clock 간 3 Word 순서 유지	blocks
T7	Lane Repair	Unit: lane 3 대체 · lane 8 무시 · en 0 / Pin: F7 + RED 1, FF	양쪽
T8	PRBS	Loopback 일치 → ERRCNT 0, 불일치 → 증가	smoke
E7	Gearbox	32b → C3D4, A1B2 2 Beat	blocks
E9	deskew	tap 0 통과, tap 2 지연	blocks
E11	PRBS	Reset Seed 5A 첫 Bit 1, Seed 0 Load 무시	blocks

Pass 기준: Test명 FAIL 출력 · Non-zero Exit Code · Timeout은 Fail · X 값 Data Sampling 금지 · 미구동 Pin은 0 (docs/04_verification_plan.md)

Regression 및 Coverage 결과

13 / 13 PASS · Line 87.5% · Branch 96.6% · Toggle 56.6% · commit 4feb39f, Verilator 5.052 재실행 (Build 4.2 s, Sim Time 511 ns)

sim/regress.log

```
[T7] unit PASS
[SMOKE] PASS
[E9] PASS
[T1] PASS
[E11] prbs PASS
[T4] PASS
[T3] PASS
[T5] PASS
[T7] PASS
[T8] PASS
[T2] PASS
[E7] PASS
[T6] PASS
[ALL] PASS
```

재현 방법: `sim/run_smoke.sh`와 동일한 Verilator 명령 (`--binary --timing`). Coverage는 `--coverage-line --coverage-toggle` 별도 Build, Waveform은 `--trace` Wrapper로 수집

Module	Line	Branch	Toggle
training_fsm	92.9%	100.0%	75.0%
csr	80.0%	100.0%	33.7%
rst_sync	100.0%	100.0%	60.0%
cmd_queue	100.0%	100.0%	45.1%
scheduler	90.0%	83.3%	29.5%
bank_fsm	90.0%	100.0%	21.3%
phy_tx	83.3%	100.0%	81.8%
phy_rx	66.7%	87.5%	60.8%
dbi	100.0%	100.0%	95.7%
lane_repair	100.0%	100.0%	93.8%
gearbox	100.0%	100.0%	51.6%
deskew	100.0%	75.0%	93.0%
cdc_fifo	100.0%	100.0%	31.0%
prbs	87.5%	100.0%	89.2%
err_cnt	66.7%	100.0%	21.4%
Total	87.5%	96.6%	56.6%

음영 = 미도달 Line/Branch 존재 Module · Module별 Instance 합산 (Block TB 포함) · Total에는 표 외
top · mc_top · dft_top · loopback · phy_top · phy_clk 포함

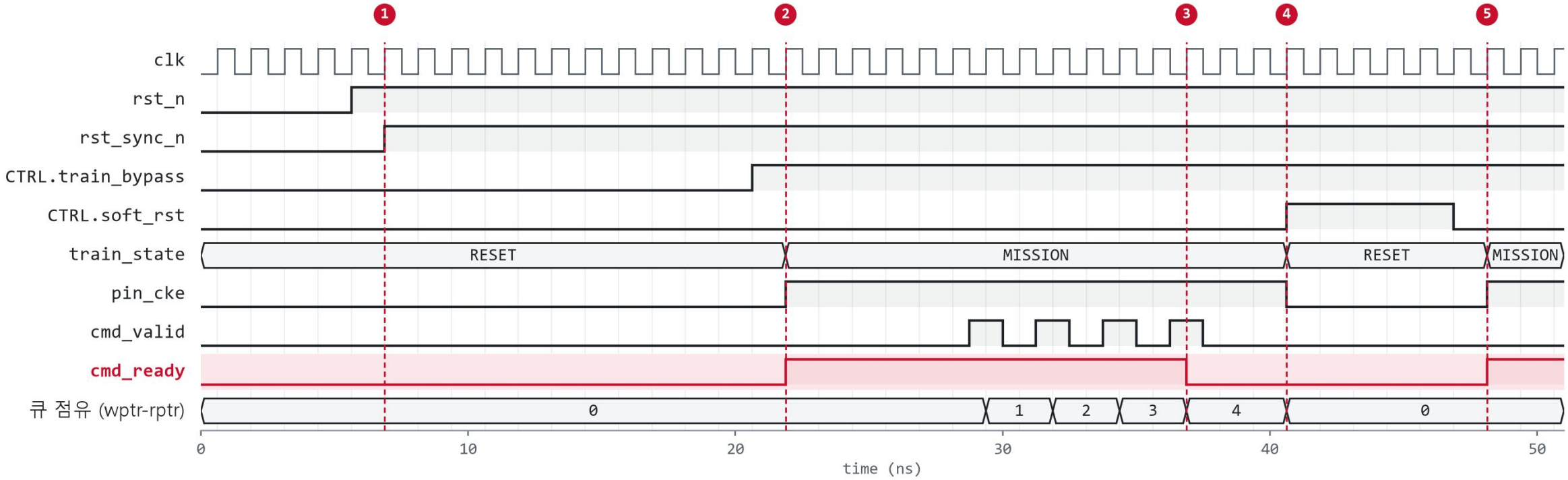
Coverage Hole 및 보완 Test

- `csr` ERRCNT · TIMING0/1 Write 미수행 → Timing 재설정 · ERRCNT Clear Test
- `err_cnt` Clear 경로 미수행 → 동일 Cycle Clear vs Error 우선순위 확인
- `prbs` Non-zero Seed Load 미수행 (On-chip load = 0)
- `scheduler` REF · Reserved Opcode, tCCD · tWTR ≤ 1 Branch 미도달
- `phy_rx` Pin DBI = 1 수신 · Loopback RX Response(rd_start & loopback) 미수행 → DBI · Loopback Read Test
- `phy_tx` PRE Pin Frame, `deskew` delay_sel > 4 Clamp 미수행
- Toggle 56.6%: Data · Address 상위 Bit 고정 → Random Pattern 필요

Functional Coverage(covergroup) 미적용

Reset 해제 · Mission 진입 · Queue Back-pressure · Soft Reset 복귀

tb_hsl_smoke 0–51 ns · Verilator 5.052 `--trace` VCD 기반 실측 Waveform · 신호명은 RTL Hierarchy 기준



1 6.9 ns Pin `rst_n` 해제 후 2-FF 동기화 → `rst_sync_n` 1 (해제 시점이 clk Edge와 중첩, p.19 B)

2 21.9 ns `train_bypass` Write → 다음 Cycle MISSION · `pin_cke` 1 · `cmd_ready` 1

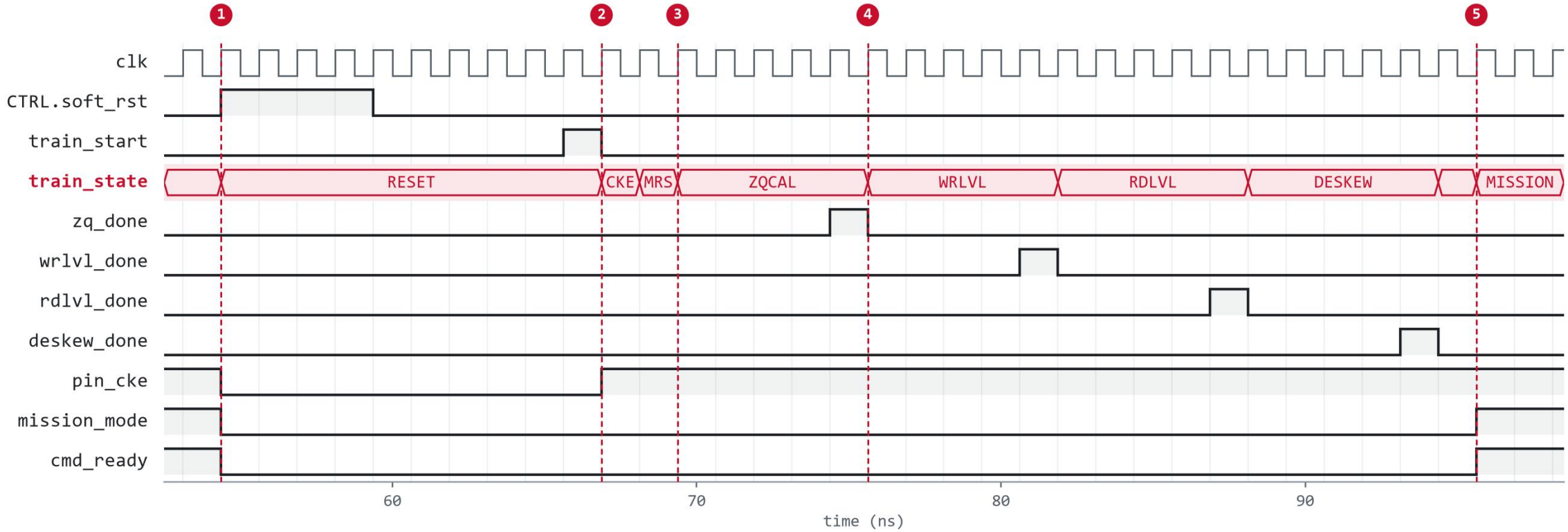
3 36.9 ns Push 4회로 Queue Full → `cmd_ready` 0. `phy_bypass` = 0 으로 PHY ready 0, Pop 없음

4 40.6 ns `soft_rst` → RESET · CKE 0 · Queue Flush. CSR 유지 (Bypass 1 유지)

5 48.1 ns `soft_rst` 해제 → Bypass 유지로 즉시 MISSION, `cmd_ready` 1

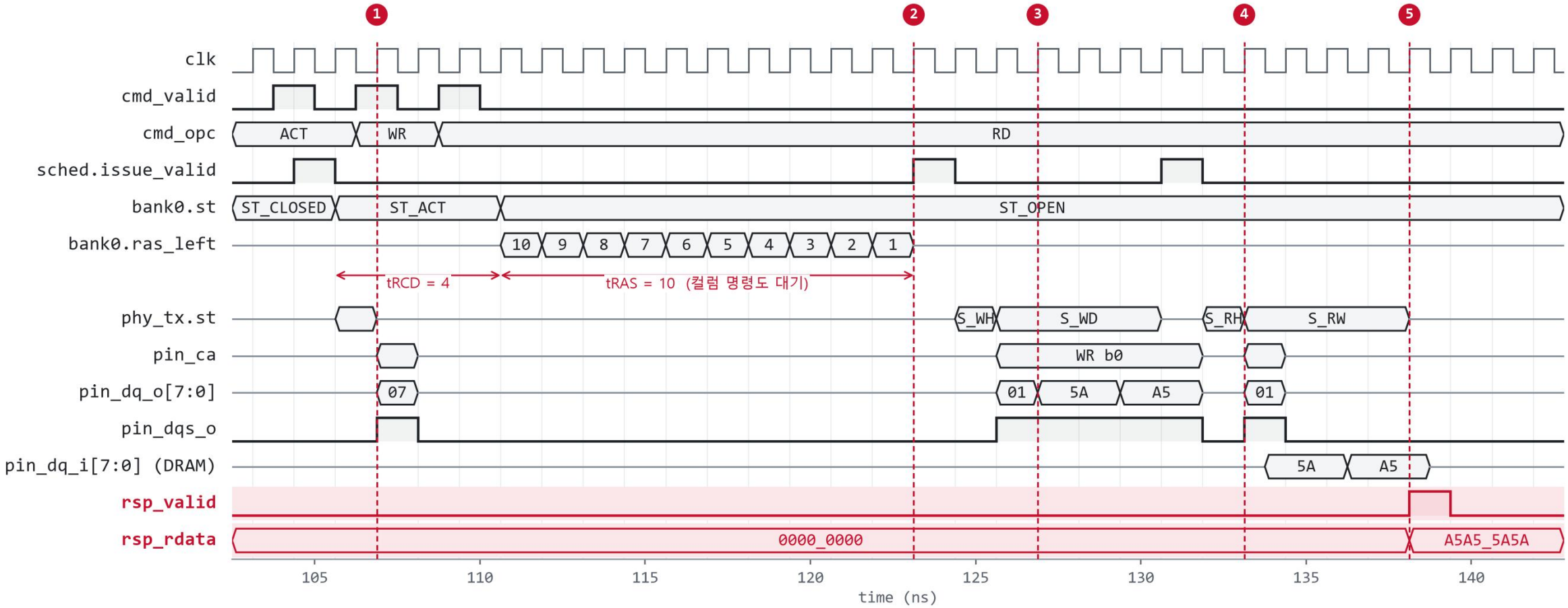
Training Sequence 동작 확인

done Pulse마다 1 State 진행, MISSION 전 Command 차단 · 각 대기 State 3 Cycle 이상 유지 확인 후 done을 1 Cycle Pulse로 인가



1. 54.4 ns CTRL = soft_rst Only → RESET, Bypass 0. T1 시작
2. 66.9 ns `train_start` Pulse → CKE State, `pin_cke` 1 → MRS 1 Cycle
3. 69.4 ns ZQCAL은 `zq_done`까지 State 유지. done 전 WRLVL 진입 시 T1 Fail
4. 75.6 ns done Pulse 1회당 1 State 진행 — WRLVL → RDLVL → DESKEW
5. 95.6 ns WRTRAIN 1 Cycle(Label 생략) 후 MISSION → `mission_mode` · `cmd_ready` 1

ACT / WR / RD Channel Traffic 및 Read Data 정합성 확인

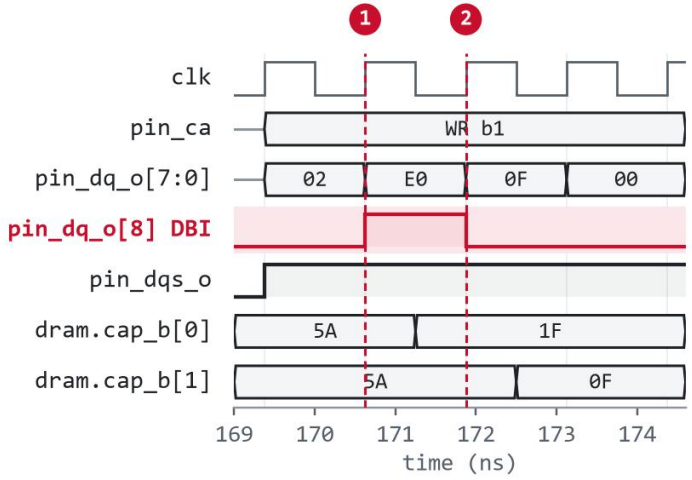


1. 106.9 ns ACT Pin Frame: `pin_ca` ACT, DQ = Row 07. Bank0 ST_ACT, tRCD 4
2. 123.1 ns `ras_left` 0 이후 WR Issue — ACT Issue 후 15 Cycle
3. 126.9 ns WR Header(Col 01) 후 Beat 5A 5A A5 A5 (동일 값 연속은 1구간 표기)
4. 133.1 ns PHY S_IDLE 복귀 후 RD Issue → RD Header, DRAM Model 이 `pin_dq_i` 구동
5. 138.1 ns `rsp_rdata` = A5A5_5A5A 일치. RD Issue → Response 6 Cycle

DBI 반전 · Lane Repair · PRBS Error Count 동작 확인

tb_hsl_smoke 동일 VCD에서 Test별 구간 발췌 · Bus 값은 16진수 표기

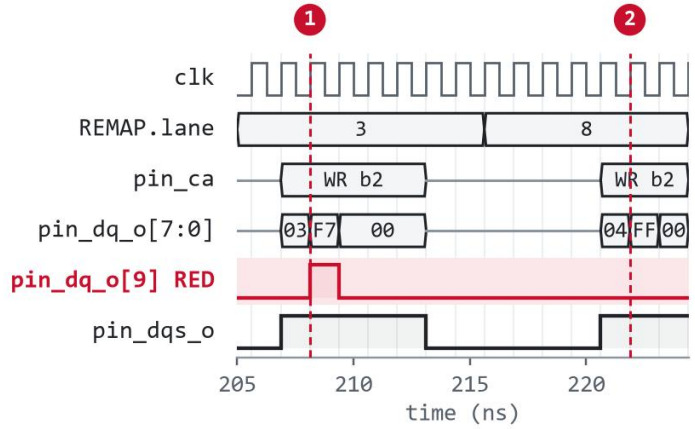
T5 · DBI-DC



1. 170.6 ns Beat0 `0x1F` ('1' 5개) → Pin `E0` + DBI 1
2. 171.9 ns Beat1 `0x0F` (4개) → 유지, DBI 0

DRAM Model에서 DBI Decode 후 cap_b = 1F · 0F 저장. Beat0 High Pin 5 → 4 (E0의 3 + DBI)

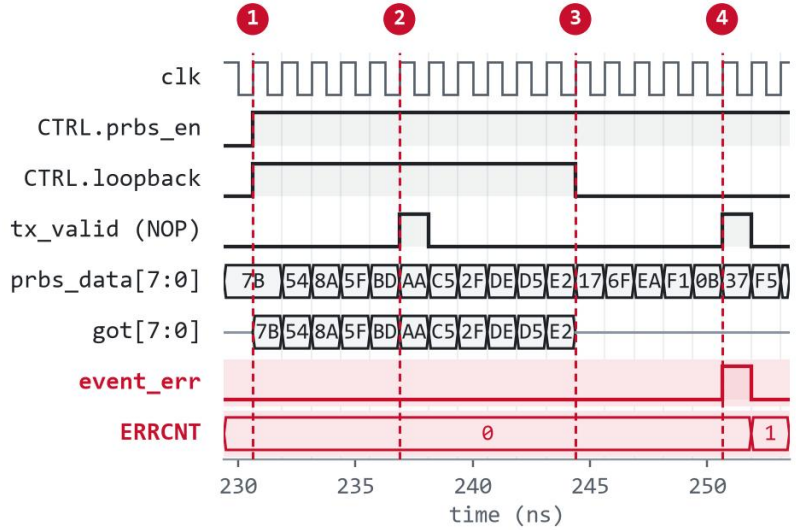
T7 · Lane Repair



1. 208.1 ns REMAP Lane 3 → Pin `F7` (bit3 → 0), RED = 원래 bit3 = 1
2. 221.9 ns Lane 8 → 대체 없음, Pin `FF` · RED 0

TX Pin Level까지만 확인 — DRAM Model과 u_rx는 RED 미사용으로 복원 경로 미검증

T8 · PRBS Parallel Loopback



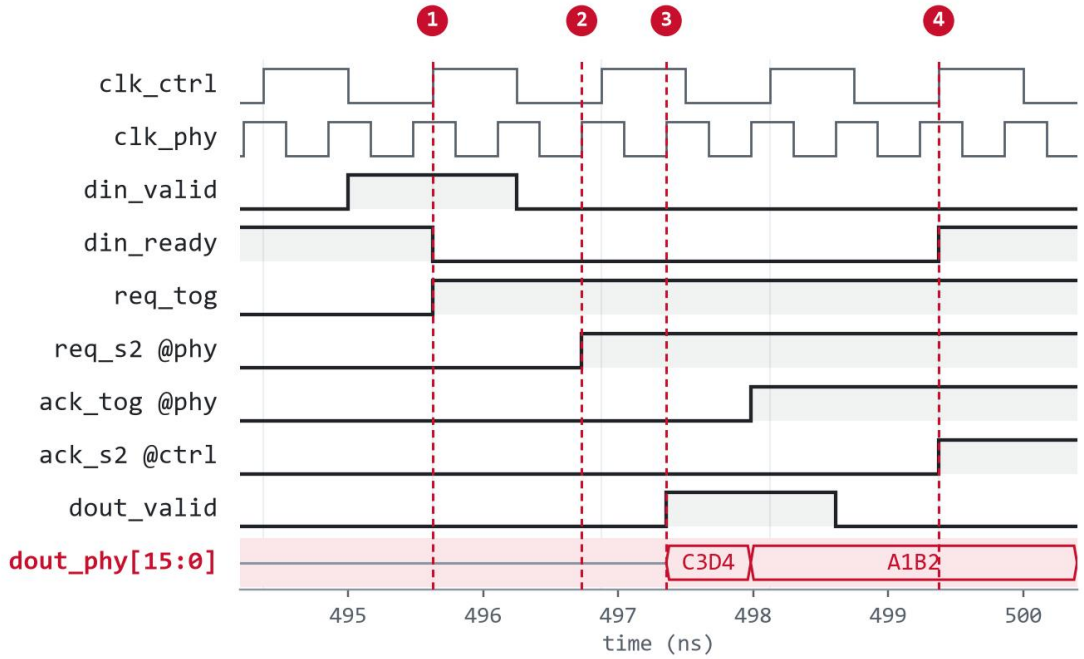
1. 230.6 ns `prbs_en` · `loopback` → `got` = `prbs_data`
2. 236.9 ns NOP Issue Cycle(`tx_valid`)에서 일치 → `event_err` 0
3. 244.4 ns `loopback` 0 → `got` = 0
4. 250.6 ns 불일치 → `event_err` → ERRCNT 1

표시는 하위 8 bit, Compare는 32 bit 전체

Gearbox 및 Async FIFO 동작 확인

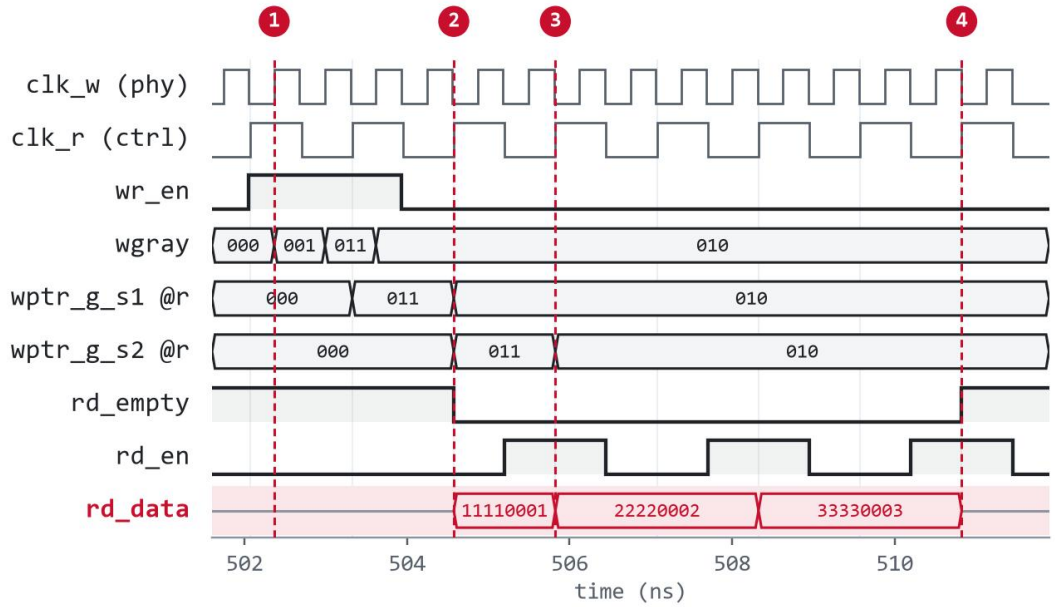
tb_hsl_blocks 독립 Instance (E7 · T6) · On-chip Gearbox · FIFO는 입력 Tie-off로 본 Block TB 결과를 동작 근거로 사용

E7 `hsl_gearbox` · 32b Word → 16b Beat × 2



1. 495.6 ns `din_valid` 수락 → Word Hold, `req_tog` Toggle, `din_ready` 0
2. 496.7 ns PHY Clock 2-FF 후 `req_s2` 반영
3. 497.4 ns 하위 `C3D4` → `A1B2` 2 Beat 출력 후 `ack_tog` Toggle
4. 499.4 ns ctrl측 `ack_s2` → `din_ready` 1, 다음 Word 수락 가능

T6 `hsl_cdc_fifo` · Write clk_phy → Read clk



1. 502.4 ns `wgray` 000 → 001 → 011 → 010, 매 Step 1 bit만 변화
2. 504.6 ns `wptr_g_s2` 도착 → `rd_empty` 0. s1이 001을 Skip해도 유효 값
3. 505.8 ns부터 Pop: 11110001 → 22220002 → 33330003
4. 510.8 ns 세 번째 Pop 후 `rd_empty` 1

Debug 이력 및 Issue 분석

좌: 기존 Issue 6건 (docs/ 기록) · 우: AI Agent 보조 재실행 · 합성 · Code Review로 도출한 추가 Issue 5건 (A · B · E는 commit edb139a에 반영)

기존 Issue — Root Cause · Action

Issue	Root Cause	Action · 참조
E3 · `cmd_ready` 기대값	Stub Queue가 `push_ready`를 0으로 고정해 MISSION에서도 ready 0	Queue 구현 후 기대값을 'Reset 직후 · Full 시 0'으로 수정, 사유를 TB 주석에 기록 (e0_e1_notes)
E12 · PHY 주기 1248 ps	반주기를 정수 312 ps로 설정 시 4구간 합 1248 ps ≠ 1250	`#312.5ps` = PHY_PERIOD_PS/2, 문서 · pkg · TB · SDC Cross-check (e12_clocks)
E1 · Soft Reset 자체 Clear	CSR까지 Soft Reset 적용 시 CTRL bit 0이 다음 Cycle에 Clear	CSR은 `rst_sync_n`만, 그 외는 `rst_logic_n` (05_pd_guide)
E8 · CDC Pointer Constraint	False Path 적용 시 해당 경로 Delay가 STA 대상에서 제외됨	Destination 주기 `set_max_delay` 1.25 / 0.625 ns, False Path 금지
E11 · PRBS · ERRCNT	Generator 0끼리 비교 시 Fault 통과, 16b Wrap 시 0으로 복귀해 정상처럼 보임	Seed 0 무시 · Reset Seed 5A, Counter FFFF Saturation (06_dft_plan)
E12 · I/O delay 4.0 ns	v0.1의 10 ns Clock 기준 40% 값 잔존	800 MHz 주기의 40% = 0.5 ns로 수정

재검증 추가 Issue (commit 4feb39f 기준)

A Timescale 정밀도 부족: `#312.5ps` → 313 ps

1 ps 정밀도로 `u_phy_clk` 구간 313 · 313 · 313 · 311 ps, Block TB `clk_phy`는 626 ps (VCD 실측). 조치: `timeprecision 100fs`

B TB Reset 해제 Race

`rst_n`을 posedge 직후 해제 → 동일 Edge에서 `sync_r[0]` = 1, `rst_sync_n`이 2 Cycle이 아닌 1 Cycle 후 해제 (5.625 → 6.875 ns). 조치: negedge 해제

C 합성 후 SDC 대상 Cell 소멸

Gearbox · Deskew · FIFO 출력이 `unused_path`로만 연결되어 Yosys에서 제거, 이어 `u_phy_clk`도 제거 → Generated Clock · FIFO max_delay 대상 부재

D 미지원 Opcode에 의한 Queue Stall (Code Review)

REF · Reserved Opcode가 Head 도달 시 `head_ok` = 0 → 영구 Stall. Simulation 미수행 (scheduler 51행 미도달). 개선안: Push 단계 거절

E Non-reset Flop 200개

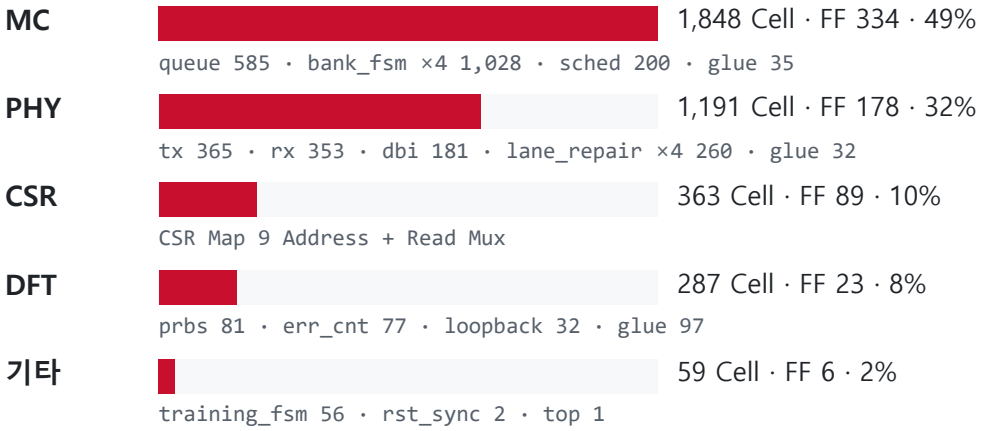
Queue `mem_*` 4 × 50b가 Reset 분기에 없음 → `$_DFFE_PP_` 200. 출력은 Empty Mask로 안전하나 DFT 규칙과 충돌. 조치: Data Array 분리 · 규칙 정비

※ SDC 크로싱 제약은 d1f8e2b에서 수정 → p.22

합성 기반 Area 분포 및 Timing Constraint

Yosys 0.52 Generic Cell 기준 Area Proxy · Cell Library · μm² · mW · Slack은 미확보 수치로 기재하지 않음

Block별 Generic Cell · FF (Hierarchical 합성, Instance 수 반영)



합계 3,748 Cell · FF 630 (Hierarchical) · Flatten 3,492 Cell · FF 630 (`$scopeinfo` 25 제외, Chip Level 상수 전파 후)

On-chip에서 제거된 clk_phy Block 단독 합성: gearbox 96 / FF 72 · cdc_fifo 378 / 146 · deskew 126 / 36

합성 전처리는 Scratch 사본에서만 수행: `import hsl_pkg::*` → localparam 치환 (Yosys 미지원), `hsl_phy_clk` Black Box, `hsl_dbi`의 `always_comb` + Module Scope `integer bi` → Latch 판정으로 `always @*`

`syn/constraints/hsl_base_die_lite.sdc` L8, L12–15, L22, L56–62

```
 8  create_clock -name clk -period $ctrl_period [get_ports clk]   (1)

12  create_generated_clock -name clk_phy \                        (2)
13    -source [get_ports clk] \
14    -multiply_by 2 \
15    [get_pins u_phy/u_phy_clk/clk_phy]

22  set_false_path -from [get_ports rst_n]                        (3)

56  set_max_delay $ctrl_period \                                  (4)
57    -from [get_cells u_phy/u_fifo/wptr_g_s1*] \
58    -to   [get_cells u_phy/u_fifo/wptr_g_s2*]
59
60  set_max_delay $phy_period \
61    -from [get_cells u_phy/u_fifo/rptr_g_s1*] \
62    -to   [get_cells u_phy/u_fifo/rptr_g_s2*]
```

1. `clk` 1.25 ns = 800 MHz, E12 Cross-check와 일치
2. `clk_phy` = clk × 2, PLL 위치의 Generated Clock
3. `rst_n` False Path는 v0.1 단순화 — Recovery/Removal Check 필요
4. Pointer 동기화는 False Path 대신 Destination 주기 `set_max_delay`

Performance (T4 Waveform 실측)

ACT → WR 15 Cycle · RD → Response 6 Cycle. Bypass Framer는 ctrl Cycle당 1 Beat (WR 5 Cycle) — Target Bandwidth 경로 아님

Power 개선 요소 (mW 미측정)

DBI-DC로 DQ+DBI 9 Pin 중 High ≤ 4 · CKE는 RESET에서만 0 · Burst 구간 외 `pin_dq_oe` = 0

PD Guide (docs/05_pd_guide.md)

Deskew 4 Tap = 8 UI Skew 허용 · SRAM Macro 없음 · PHY Pin Logic은 Bump 측, CSR · FSM은 내측 배치

DBI-AC 추가: 전이 규칙, beat당 2:1 mux 체인, off · DC · AC 비교

CSR CTRL[6]로 DC · AC 선택 · AC는 직전 UI에 나간 바이트 대비 DQ 전이가 4개를 넘으면 반전 → UI당 DQ 전이 ≤ 4 (commit e30ea96)

두 규칙과 beat 0의 기준

- **DC**: beat의 1이 4개 초과 → 반전. high 핀 수를 줄임 (high에서 종단 전류가 흐르는 I/O)
- **AC**: 직전 UI 대비 DQ 전이 4개 초과 → 반전. 전이 수를 줄임 (종단 없는 짧은 채널의 스위칭 전력 · SSO)
- **beat 0**: 바로 앞 UI와 비교. bypass framer에서는 WR 헤더(DQ[3:0] = col)가 그 UI

rtl/phy/hsl_dbi.sv — AC 체인을 mux 하나로

```
// 직렬: beat마다 XOR → popcount → 비교 → 반전이 이어짐
r[b]   = pop8(beat[b] ^ beat[b-1]);   // 4 beat 병렬, beat[-1] = prev
inv    = inv ? (r[b] < 4) : (r[b] > 4); // b-1 반전 시 전이 = 8 - r
ac_inv[b] = inv;
// HEAD: AC_SERIAL = 1이면 직렬 구조 (블록 TB가 두 구조 등가 확인)
```

hsl_dbi · Nangate45 typical (syn/ppa_block.sh)

구조	면적	Cell	입력 → 출력 지연
DC만 (4d38ae9)	130 μm²	101	0.449 ns
DC + AC, mux 체인 (기본)	289 μm²	224	**0.882 ns**
DC + AC, 직렬 (AC_SERIAL = 1)	301 μm²	229	1.822 ns

직렬은 블록 하나로 컨트롤러 주기 1.25 ns를 넘는다. mux 체인은 0.37 ns를 남기지만 칩 경로(큐 head → PRBS 먹스 → DBI → 리페어)는 칩 STA에서 다시 확인.

UI당 전이 · high 핀 수 (DQ + DBI, T9 측정)

데이터 (측정 위치)	지표	off	DC	AC
PRBS7 · WR 254개 (칩 핀)	전이	4.01	4.08	**3.35 (−16%)**
	high	4.03	**3.31 (−18%)**	4.13
55 AA 55 AA · WR 254개 (칩 핀)	전이	7.00	7.00	**1.64 (−77%)**
랜덤 연속 스트림 (블록)	전이	4.00	4.17	**3.38**
40,000 UI	high	4.01	**3.27**	4.36

- **DC는 high를, AC는 전이를 줄인다**. 랜덤에 가까운 데이터에서 각 규칙은 다른 쪽 지표를 조금 늘린다
- **전이가 몰린 데이터에서 AC가 크다**. 55/AA에서 DC는 아무것도 못 하고(beat마다 1이 4개), AC는 DQ를 고정하고 DBI만 바꾼다
- **이론값과 일치**. 랜덤 스트림이 이항분포 기대값(off 4.00 · DC 4.16 · AC 3.37)과 ±0.05 안

T9 · 회귀 15/15 PASS (Verilator 5.052 · ModelSim 10.1d)

- **블록**: 직렬 참조 모델과 랜덤 4,000 버스트, 두 AC 구조 등가, 동점(전이 정확히 4) directed
- **칩**: AC 핀 값 정확 비교, PRBS7 · 55/AA 스트림에서 규칙 위반 0
- **읽기**: 채널 모델이 RD 데이터에 DBI(DC · AC)를 붙이고 RX가 복원, 32 RD 일치. 핀 DBI = 1 수신 구멍 해소

10 Update ② Review Fixes

리뷰 반영: CDC Gray 크로싱 · SDC · 루프백 RD · ModelSim 빌드 (d1f8e2b)

p.10의 FIFO 코드와 p.20의 SDC는 수정 전(4feb39f) 상태 · HEAD는 Verilator 5.052와 ModelSim-Altera 10.1d 모두 15/15 PASS

① CDC: Gray 포인터를 플롭에서 넘김

- **문제**: 싱크러나이저가 binary 플롭의 XOR(조합 Gray)을 바로 받음. 여러 비트가 바뀌는 순간의 glitch를 첫 싱크 플롭이 잡을 수 있음
- **수정**: wgray · rgray를 소스 클럭 플롭으로 두고 그 플롭만 넘김. full / empty 타이밍은 그대로

rtl/phy/hsl_cdc_fifo.sv

```
// 4feb39f: 조합 Gray가 넘어감
assign wgray = bin2gray(wbin);

// HEAD: 소스 클럭의 Gray 플롭
always_ff @(posedge clk_w ...)
  wbin  <= wbin_next;
  wgray <= bin2gray(wbin_next);

// clk_r 쪽 2-FF 싱크
wptr_g_s1 <= wgray;  // crossing
wptr_g_s2 <= wptr_g_s1;
```

② SDC: 크로싱 자체를 제약

- **문제**: set_max_delay가 한 클럭 안의 *_s1 → *_s2를 묶고 있어, 실제 크로싱(소스 → *_s1)은 제약 밖
- **수정**: 소스 Gray 플롭 → 첫 싱크 플롭에 빠른 쪽 주기 0.625 ns. 한 Gray 단계의 비트가 한 주기 안에 도착. false path 아님

syn/constraints/hsl_base_die_lite.sdc

```
# 4feb39f: 한 클럭 안의 경로
set_max_delay $ctrl_period \
  -from u_fifo/wptr_g_s1* \
  -to   u_fifo/wptr_g_s2*

# HEAD: wgray (clk_phy) -> s1 (clk)
set_max_delay $phy_period \
  -from u_phy/u_fifo/wgray* \
  -to   u_phy/u_fifo/wptr_g_s1*
# rgray (clk) -> rptr_g_s1 (clk_phy)
#   같은 형태
```

③ 루프백 RD · ModelSim 빌드

- **루프백 RD**: 루프백 중 RD가 0이나 다음 큐 head의 데이터를 돌려줌(요구 5 미충족) → 마지막 루프백 WR 버스트를 lb_hold에 두고 RD로 반환
- **ModelSim 10.1d**: 폭이 파라미터인 캐스트 BANK_W'(gi)를 파싱 못 함 → 타입이 있는 localparam. 이전의 ModelSim 통과 기재를 바로잡음

HEAD 회귀 (Verilator 5.052 · ModelSim 10.1d)

```
[T7] unit PASS      [SMOKE] PASS
[E9] PASS           [T1] PASS
[E11] prbs PASS     [T4] PASS
[T3] PASS           [T5] PASS
[T7] PASS           [T8] PASS
[T2] PASS           [E7] PASS
[T6] PASS           [T9] unit PASS
[T9] PASS
[ALL] PASS

// T8: 채널 모델에 다른 값이 있는 col을 읽어 lb_hold 반환 확인
```

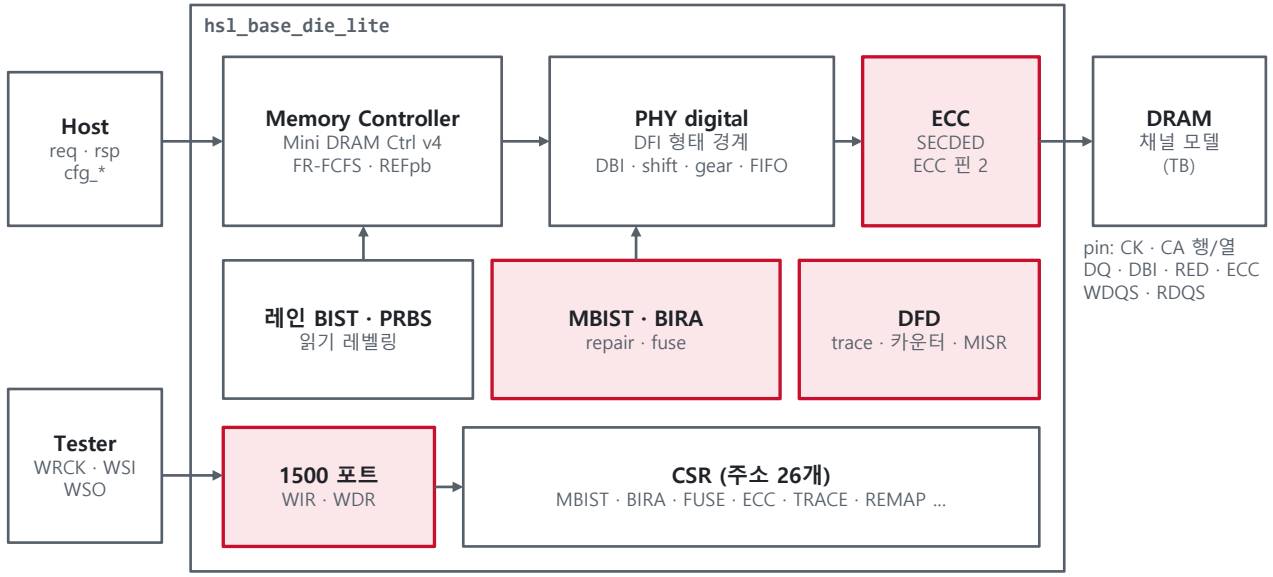
세 건의 공통점

모두 시뮬레이션은 통과한 상태에서 **설계 의도**(CDC 안전성, 제약이 걸리는 경로, 요구 5)와 **도구 호환성**을 따로 검토해 찾았다. 동작 검증과 별개로 코드 · 제약 리뷰 단계를 두고, 고친 항목마다 회귀 테스트를 추가했다(T8 루프백 RD).

DFT / DFD 확장: 배열 시험 · 수리 · ECC · 관측 · 시험 포트

빨간 테두리 = 이번에 더한 블록 · 시험 장비는 1500 포트 하나로 모든 단계를 돈다 (commit f018966 → 14fec6e, 쪽마다 측정 commit)

칩 구조 (14fec6e) · 컨트롤러 교체와 mission PHY 상세는 README



23 / 23 PASS
SMOKE · T1–T17 · E7 · E9 · E11, Verilator 5.052 · ModelSim 10.1d

53,316 µm² · 4,808 FF
Nangate45, 배선 없음 · DFT 17% · DFD 11% · ECC 4%

Line 95.5% · Branch 91.4%
Toggle 78.8% · rtl/ 전체, 인스턴스 병합

HSL 로직 STA 통과
800 MHz · 1.6 GHz · RDQS. 컨트롤러만 459 MHz

시험 흐름 (docs/08_test_flow.md) · 단계마다 회귀 테스트 하나

#	단계	합격 조건	실패하면
0	전원 · 리셋	fuse 4워드 SECDED 디코드 → repair 표, 정정 불가 0	fuse bin
1	접근	1500 ID HSL1 · VERSION 0x300	접근 bin
2	디지털 데이터패스	PRBS7 병렬 루프백 오류 0	로직 bin
3	링크 핀	레인 BIST. 한 레인 실패 → REMAP 뒤 재검사 PASS	두 레인 이상 → 링크 bin
4	읽기 레벨링	모든 레인 eye 폭 2 코드 이상	링크 bin
5	ECC 디코더	self-test 78개, 주입대로 정정 · 정정 불가	로직 bin
6	배열 (ECC off)	블록마다 March C- 실패 0	→ 7 수리
7	수리	BIRA → apply → 재시험 PASS → fuse program	수리 불가 → 배열 bin
8	hard repair 확인	리셋 → fuse 로드 → 재시험 PASS	fuse bin
9	mission	ECC on 트래픽, EVCNT · MISR 기대값	FA: ECC_ADDR · trace

- **3이 6보다 먼저**: DQ 핀 stuck-at-1이면 모든 r0이 실패한다(T13). 링크를 먼저 고쳐야 배열 고장과 구분된다
- **6은 ECC off**: 켜면 1 bit 배열 고장이 정정되어 보이지 않는다(T15의 stuck bit)
- **시험 시간**: March C- 연산당 13.2클럭(ffc073d 측정) → 칩 배열 전체 약 2.16M클럭 = 2.7 ms(추정)

March MBIST: 컨트롤러를 거쳐 한 연산씩, 참조 모델과 같은 검출표

hsl_mbist · bank 하나, 최대 16 row × 16 col · MATS+ / March X / March C- · CSR MBIST · MBIST_LOG · MBIST_BITS (commit f018966)

알고리즘 · 8워드 실행 (T13)

알고리즘	요소	연산/워드	8워드
MATS+	{⇳ (w0); ⇑ (r0,w1); ⇓ (r1,w0)}	5	507클럭
March X	{⇳ (w0); ⇑ (r0,w1); ⇓ (r1,w0); ⇳ (r0)}	6	616클럭
March C-	{⇳ (w0); ⇑ (r0,w1); ⇑ (r1,w0); ⇓ (r0,w1); ⇓ (r1,w0); ⇳ (r0)}	10	1,021클럭

연산을 하나씩 내는 이유

- 컨트롤러는 다른 주소의 순서를 바꾼다(FR-FCFS, 쓰기 배치). 여러 개를 띄우면 aggressor 쓰기가 victim 읽기보다 늦게 배열에 닿아 coupling fault를 놓친다
- 쓰기는 응답 → q_empty(WR이 핀으로 나감) → 4클럭, 읽기는 응답을 기다린다. 연산당 약 13.2클럭 (March C- 64워드 8,443클럭, ffc073d)
- 실제 HBM의 배열 MBIST는 core die에서 배열을 직접 몬다. 여기서는 base die → 핀 → 배열 경로 전체를 시험한다

STA와 리뷰가 바꾼 것

- 첫 구현은 주소를 조합으로 만들었다(가변 shift · row 덧셈 · XOR 맵, 0.78 ns). 컨트롤러 req_ready 앞에 붙어 −0.749 ns, 실패 끝점 1,752개 → S_LOAD에서 한 클럭 먼저 계산해 플롭에서 냄
- 레인 BIST와 포트 공유: 다른 엔진이 busy일 때 start를 버리면 읽기 레벨링이 멈춘다 → start를 잡아 두었다가 포트가 비면 냄, 레인 BIST 우선
- 면적 2,849 μm² · FF 264 (실패 로그 16 × 8 bit와 중복 비교기 16개가 큰 몫)

배열 고장 24개 검출 (T13 = tb/march_ref.py 참조)

고장 (victim bit 3, aggressor bit 7)	개수	MATS+	March X	March C-
SA0 · SA1	2	2	2	2
TF ↑ · ↓	2	1	2	2
CFin (전이 × aggressor 앞 · 뒤)	4	3	4	4
CFid (전이 × 강제 값 × 순서)	8	3	4	8
CFst (상태 × victim 값 × 순서)	8	4	5	8
합계 (칩 = 참조)	**24**	**13**	**17**	**24**

- **MATS+는 TF ↓를 놓친다**: 마지막 w0 뒤에 읽기가 없다. March X는 끝의 ⇳ (r0)으로 잡는다
- **CFid · CFst의 두 순서**(aggressor가 victim 앞 · 뒤)는 두 방향 각각에 r0,w1과 r1,w0이 모두 있어야 잡힌다 → March C-

T13 (f018966, 두 시뮬레이터 같은 결과)

- 고장마다 세 알고리즘의 검출 여부가 참조와 같고, 첫 실패가 victim 워드 · 비트
- 실패 워드 로그: SA1 · SA0 · TF ↑ → 처음 실패한 순서로 03 29 3F, 실패 읽기 7
- DQ5 핀 stuck-at-1: r0 192개 실패, 로그 16개가 차고 넘침 비트, 다른 비트 20202020
- 두 엔진 start를 두 순서로: 둘 다 돌고 PASS, 함께 busy인 클럭 0

BIRA · spare · fuse: 실패 로그로 spare를 고르고 fuse에 굽기

spare row 2 + col 2 · repair 표 → 배열 spare · fuse는 SECDED (39,32), 리셋 뒤 자동 로드 (commit 6c29d60)

흐름

```
MBIST(March C-) ─▶ 실패 워드 로그 16개 ─▶ BIRA ─▶ apply
  ─▶ repair 표 ─▶ 배열 spare                    // soft repair
repair 표 ─▶ FUSE program ─▶ fuse 워드 4개 (SECDED 39,32)
칩 리셋 ─▶ fuse 자동 로드 ─▶ 디코드 ─▶ repair 표     // hard repair
```

hsl_bira: 세 단계

단계	하는 일
1. 넘친 로그	기록하지 못한 실패 워드가 있으므로 수리 불가
2. must-repair	변화가 없을 때까지 반복: 실패가 남은 spare col보다 많은 row → spare row, 반대도 같음
3. spare 순서 탐색	남은 spare의 순서(최대 16가지)마다 로그 순서로 덮어 봄. 모두 덮는 첫 순서를 씀

수리 성공률: greedy 대 순서 탐색 (16 × 16 블록, 맵 3,000개씩)

실패 워드	1 + 1	2 + 2 greedy	2 + 2 탐색 = exhaustive	3 + 3 greedy / exh.
3	31.4%	100%	100%	100% / 100%
4	4.8%	100%	100%	100% / 100%
5	0.6%	63.1%	**73.2%**	100% / 100%
6	0.2%	31.7%	**41.3%**	100% / 100%
8	0.0%	4.9%	5.2%	67.9% / 81.4%
10	0.0%	0.2%	0.2%	27.7% / 41.0%

- **첫 구현은 greedy**였다. Python 참조(tb/bira_ref.py)가 해가 있는 맵을 놓치는 것을 보여 교체. 모든 해는 spare 순서 하나에 해당하므로 해가 있으면 찾는다(1 + 1 · 2 + 2 전 맵에서 exhaustive와 assert)

soft · hard repair

- repair 표: spare마다 16 bit {valid, bank, row, col}. apply가 BIRA 결과를 넣고 clear가 비운다. 배열은 표에 있는 row · col 접근을 spare로 보낸다
- fuse program: 항목마다 SECDED 인코딩(0 → 1만). 리셋 뒤 로드: 1 bit는 정정해 세고, 2 bit는 그 항목을 버리고 FUSE[15:12]에 표시
- STA: BIRA 개수를 직렬 증가 체인으로 세면 −0.318 ns → 일치 벡터 + 균형 popcount, 레지스터 뒤 결정(항목당 3클럭). fuse 읽기 → 디코더 −0.256 ns → 먼저 레지스터

T14 (6c29d60, 두 시뮬레이터)

- A 실패 8워드: 표 AC10 AC60 AC05, 재시험 PASS / B hard repair: fuse → 표 비움 → 리셋 → 4항목 복원, 재시험 PASS
- C fuse ECC: 1 bit 정정, 2 bit는 그 항목(col 5)을 버려 3워드가 다시 실패
- D 수리 불가(대각선 5워드, 넘친 로그) / E greedy가 못 고치는 맵을 row +10 · +14, col 9 · 10으로 수리
- F 랜덤 12맵: 판정 · 할당이 TB 참조와 같음, 수리 가능 9개(greedy 8) 모두 PASS
- 면적: BIRA 1,975 · repair 1,152 · SECDED 인코더 122 + 디코더 348 μm²

SECDED ECC: ECC 핀 2개, 오류 주입 · 로그, 디코더 self-test

32 bit 버스트마다 (39,32) · 검사 비트는 데이터 beat 옆 ECC 핀 2 × 4 UI, DRAM이 함께 저장 · 기본 off (commit ffc073d)

```
rtl/phy/hsl_ecc.sv 위치 (hsl_phy_mis 안)
쓰기: 원시 워드 –▶ tx_raw –(반 클럭)–▶ 인코더 –▶ ECC 핀 beat
      DBI –▶ lane shift –▶ tx_word –▶ TX 기어 (데이터 + ECC 48 bit)
읽기: RDQS –▶ Gray FIFO 48 bit –▶ DBI 복원 –▶ 레지스터 단 –▶ 디코더
      RD 추적 FIFO(CA 레지스터) –▶ 첫 오류의 호스트 주소
```

기능 · CSR ECC · ECC_INJ · ECC_CNT · ECC_ADDR

기능	동작
정정 · 검출	1 bit 정정 → ECC_CNT[15:0], 2 bit는 그대로 넘기고 [31:16]에 셈(포화)
첫 오류 로그	호스트 주소 · syndrome · 정정 여부를 clear까지 유지
오류 주입	데이터 · 검사 비트 마스크를 다음 쓰기 하나에 XOR → DRAM에 저장
디코더 self-test	깨끗한 1 + 1 bit 39 + 이웃 2 bit 38 = 78개를 데이터 경로 디코더에

STA가 정한 구조와 대가

- 인코더를 컨트롤러 쓰기 경로에 두면 –1.029 ns → 원시 워드를 레지스터로 받고 TX 기어 E1까지 반 클럭에 인코딩(clk_phy +0.044 ns)
- 디코더 앞 레지스터 단(–0.403), self-test 3단 파이프(–1.125, codeword는 읽기 반환 레지스터로 넣어 mux 없음), RD 추적은 CA 레지스터에서(–0.810)
- 대가: 읽기 지연 +1(열린 row 9 → 10). 컨트롤러가 RD를 4개만 띄워 32 RD가 76 → 97클럭. RD_PIPE를 PHY 왕복에 맞추는 것이 해결책

T15 (ffc073d, 두 시뮬레이터)

항목	결과
self-test	78개 PASS. 정정 플래그를 0에 묶으면 FAIL, 39개 틀림
깨끗한 트래픽	bank 3 row 40의 16워드 쓰기 · 읽기, 오류 0
데이터 1 bit (bit 9)	정정되어 12345678, 첫 오류 주소 1062, syndrome 4E
검사 비트 1 bit	정정, 첫 오류 기록은 그대로
2 bit (bit 0, 1)	정정 불가 1, 데이터는 FACE0003 그대로
배열 bit 17 SA0	ECC on: FFFFFFFF로 정정 / off: FFFDFFFF

한계와 면적

- SECDED라 3 bit 이상은 잘못 정정될 수 있다. DRAM 안의 on-die ECC와는 다른 층이다
- 정정 불가는 CSR로만 알린다. 컨트롤러의 DFI 쪽에 오류 입력이 없어 호스트 응답(rsp_err)에 실리지 않는다
- ECC 핀은 DBI와 레인 리페어 밖이다. enable은 컨트롤러가 idle일 때 바꾼다
- 면적 hsl_ecc 1,960 μm² · FF 153(카운터 32, self-test 파이프, 로그), SECDED 인코더 122 · 디코더 359 μm²
- fault-injection decoder BIST는 공개 자료의 개념으로 다시 짰다

DFD: trace · 카운터 · MISR, 심은 버그를 trace로 찾기

hsl_dfd · CA 핀 레지스터, 읽기 데이터, ECC · PRBS · MBIST 이벤트를 관측 · CSR TRACE · TRACE_DATA · EVCNT0/1 · MISR (commit 796d969)

기능

기능	동작
trace 16 × 32 bit	명령이나 이벤트가 있는 클럭만 기록. {ts 8, 행 CA, 열 cmd · bank, 플래그}. bank 필터
trigger · pre / post	UE · CE · PRBS · MBIST · bank 명령 · 수동. trigger 앞 칸이 다 찬 뒤에만 받고 post개 뒤 정지
명령 카운터	핀의 ACT · RD · WR · REFpb, 16 bit 포화
MISR	읽기 데이터 32 bit signature (x^32 + x^22 + x^2 + x + 1)

디버그 순서 (T16) · 버그를 모른다고 두고 칩 관측 수단만 씀

- **증상**: ECC on 랜덤 읽기 200개. 호스트 데이터는 모두 맞지만(정정) 정정 오류 32개. 첫 오류 주소 1830 = bank 3
- **좁히기**: bank 3만 읽는 트래픽 → 정정 오류가 모두 bank 3 읽기
- **trace**: trigger = ECC 정정, 필터 = bank 3, post 1. 3번 잡음
- **읽기**: 세 번 모두 REFpb b3 → 8클럭 뒤 ACT → 11클럭 뒤 첫 RD → 그 RD 8클럭 뒤 오류. REFpb 전의 RD 7개는 깨끗 → refresh 뒤 첫 읽기 가설
- **확인**: 심은 버그(bug_refdis: REFpb 뒤 16클럭 안 RD의 bit 5 반전)를 끄면 오류 0

T16 trace 덤프 (TRACE_DATA, 세 번 중 하나)

```
t= 33  RD b3  data
t= 35  RD b3  data
t= 37  RD b3
t= 40  RD b3  data       // REFpb 전 RD 7개: 오류 없음
t= 46  RD b3
t= 49  PRE b3  data
t= 52  REFpb b3
t= 60  ACT b3 r60        // +8 = tRFCpb
t= 63  RD b3             // REFpb +11
t= 65  RD b3
t= 67  RD b3
t= 69  RD b3
t= 71  ECC-CE  <- trigger // 첫 RD +8 = 핀 RD → 디코더 플래그
t= 72  RD b3  data
```

면적 · 한계

- 면적 6,019 μm² · FF 701. 거의 trace 512 bit 플롭이다. 실제 칩이라면 trace는 SRAM 매크로
- trace 항목에 column 주소가 없다. 이벤트는 카운터 변화로 보므로 ECC 이벤트가 핀 RD보다 몇 클럭 늦게 찍힌다
- trigger 중 UE · PRBS · MBIST · bank 명령은 시험하지 않았다 (hsl_dfd line 75%)

IEEE 1500 방식 시험 포트, STA가 바꾼 DFT · ECC RTL

hsl_w1500: WIR · WBY · ID · 41 bit CSR WDR, WRCK를 clk로 오버샘플 (commit 14fec6e) · STA: Nangate45 + OpenSTA 3.1

WIR (3 bit, 리셋 BYPASS) · 코드와 배치는 이 설계가 정함

WIR	레지스터	동작
000 BYPASS	WBY 1 bit	한 비트 늦춰 통과 (그 외 코드도)
001 ID	32 bit	capture에서 48534C31 (HSL1)
010 CSR	41 bit {we, addr 8, data 32}	update에서 쓰거나 읽음. 읽기는 scan 두 번

설계 선택과 T17

- **오버샘플**: 입력 7개를 2-FF 동기화하고 WRCK 엣지를 clk에서 찾는다. WRCK ≤ clk / 8 → STA에 새 클럭도 CDC도 없다. 대가는 WRCK 속도 상한
- **CSR 전체 접근**: 한 WDR로 MBIST · BIRA · repair · fuse · ECC · trace · REMAP에 닿는다. 접근하는 1–2 클럭 동안 CSR 버스를 잡는다
- **리셋**: rst_sync_n에 둬서 soft reset에도 WIR 유지. WRSTN이 BYPASS로 되돌린다
- **T17**: ID HSL1, WBY B2 → 64(한 비트 지연), CSR 읽기, March C-를 직렬 포트만으로 시작해 폴링 두 번(82 bit씩)에 PASS
- **없는 것**: WBR · EXTEST. DBI · RED 핀 고장 시험(레인 BIST로 안 보임)이 남았다
- 면적 1,369 μm² · FF 170 (shift · update 레지스터, 읽기 보관, 동기화기)

STA가 짚어 고친 경로 (모두 800 MHz 통과로 바뀜)

경로	slack	조치
MBIST 조합 주소 → 컨트롤러 req_ready	−0.749	한 클럭 먼저 계산, 플롭에서 냄
BIRA 항목 → 개수 직렬 체인 → 결정	−0.318	균형 popcount, 레지스터 뒤 결정
fz_rdata → SECDED 디코더 → repair 표	−0.256	fuse 읽기 데이터 먼저 레지스터
쓰기 데이터 → ECC 인코더 → TX 워드	−1.029	원시 워드 레지스터 + 반 클럭 인코딩
읽기 FIFO → DBI → 디코더 → rddata	−0.403	디코더 앞 레지스터 단
self-test → 디코더 → 실패 카운터	−1.125	세 단계 파이프라인
ccmd → 루프백 판정 → FIFO pop	−0.810	RD 추적 · 루프백을 CA 레지스터에서

칩 STA 결과 (14fec6e)

클럭 · 검사	결과
clk_phy 1.6 GHz	+0.044 ns (최악: ECC 인코더 반 주기)
rdqs 0.625 ns · FIFO 크로싱	+0.106 ns · Gray 포인터 0.104 / 0.625 ns
clk 800 MHz, HSL 로직	모두 통과
clk 800 MHz, 컨트롤러	−0.930 ns, 2.18 ns = 459 MHz
원인 → 방향	선택 + 타이머 갱신 한 클럭 루프 → DFI 1 : 2

11 Summary

핵심 요약 및 향후 과제

Spec → Architecture → RTL → Verification → PD Guide 전 과정 결과와 미해결 항목 정리

핵심 요약

Spec 수치를 문서 → `hsl_pkg` → RTL → TB → SDC까지 단일 값으로 일관 관리하고,
그 정합성을 Simulation · Waveform · 합성 결과로 검증

향후 과제

- 컨트롤러 DFI 1 : 2 (400 MHz, 클럭당 명령 슬롯 2개) — 지금 459 MHz에서 닫힘
- RD_PIPE를 PHY 왕복에 맞춤 — ECC로 늘어난 읽기 지연 회수 (32 RD 97클럭)
- 1500 WBR · EXTEST로 DBI · RED 핀 시험, 로직 스캔 · ATPG
- deskew를 읽기 경로에 연결, 시험하지 않은 DFD trigger 소스 검증
- p.3–20을 HEAD 구조로 갱신 (컨트롤러 교체 · mission PHY는 지금 README에만)

권용현

010-6408-8646 · kyongh0331@naver.com

github.com/yhkwon-git/High_Speed_Link_Lite